

15.4 External-Memory Sorting

In addition to data structures, such as maps, that need to be implemented in external memory, there are many algorithms that must also operate on input sets that are too large to fit entirely into internal memory. In this case, the objective is to solve the algorithmic problem using as few block transfers as possible. The most classic domain for such external-memory algorithms is the sorting problem.

Multiway Merge-Sort

An efficient way to sort a set S of n objects in external memory amounts to a simple external-memory variation on the familiar merge-sort algorithm. The main idea behind this variation is to merge many recursively sorted lists at a time, thereby reducing the number of levels of recursion. Specifically, a high-level description of this **multiway merge-sort** method is to divide S into d subsets $S_1, S_2, \dots, S_d$ of roughly equal size, recursively sort each subset S_i , and then simultaneously merge all d sorted lists into a sorted representation of S . If we can perform the merge process using only $O(n/B)$ disk transfers, then, for large enough values of n , the total number of transfers performed by this algorithm satisfies the following recurrence equation:

$$t(n) = d \cdot t(n/d) + cn/B,$$

for some constant $c \geq 1$. We can stop the recursion when $n \leq B$, since we can perform a single block transfer at this point, getting all of the objects into internal memory, and then sort the set with an efficient internal-memory algorithm. Thus, the stopping criterion for $t(n)$ is

$$t(n) = 1 \quad \text{if } n/B \leq 1.$$

This implies a closed-form solution that $t(n)$ is $O((n/B) \log_d(n/B))$, which is

$$O((n/B) \log(n/B) / \log d).$$

Thus, if we can choose d to be $\Theta(M/B)$, where M is the size of the internal memory, then the worst-case number of block transfers performed by this multiway merge-sort algorithm will be quite low. For reasons given in the next section, we choose

$$d = (M/B) - 1.$$

The only aspect of this algorithm left to specify, then, is how to perform the d -way merge using only $O(n/B)$ block transfers.

15.4.1 Multiway Merging

In a standard merge-sort (Section 12.1), the merge process combines two sorted sequences into one by repeatedly taking the smaller of the items at the front of the two respective lists. In a d -way merge, we repeatedly find the smallest among the items at the front of the d sequences and place it as the next element of the merged sequence. We continue until all elements are included.

In the context of an external-memory sorting algorithm, if main memory has size M and each block has size B , we can store up to M/B blocks within main memory at any given time. We specifically choose $d = (M/B) - 1$ so that we can afford to keep one block from each input sequence in main memory at any given time, and to have one additional block to use as a buffer for the merged sequence. (See Figure 15.8.)

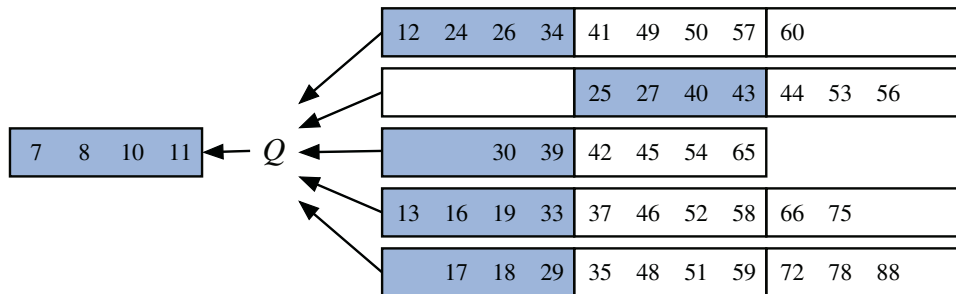


Figure 15.8: A d -way merge with $d = 5$ and $B = 4$. Blocks that currently reside in main memory are shaded.

We maintain the smallest unprocessed element from each input sequence in main memory, requesting the next block from a sequence when the preceding block has been exhausted. Similarly, we use one block of internal memory to buffer the merged sequence, flushing that block to external memory when full. In this way, the total number of transfers performed during a single d -way merge is $O(n/B)$, since we scan each block of list S_i once, and we write out each block of the merged list S' once. In terms of computation time, choosing the smallest of d values can trivially be performed using $O(d)$ operations. If we are willing to devote $O(d)$ internal memory, we can maintain a priority queue identifying the smallest element from each sequence, thereby performing each step of the merge in $O(\log d)$ time by removing the minimum element and replacing it with the next element from the same sequence. Hence, the internal time for the d -way merge is $O(n \log d)$.

Proposition 15.3: Given an array-based sequence S of n elements stored in external memory, we can sort S with $O((n/B) \log(n/B) / \log(M/B))$ block transfers and $O(n \log n)$ internal computations, where M is the size of the internal memory and B is the size of a block.